

Gratitude

Server host procedure

RHEL VM deploy checklist

September 16, 2026

This document covers only how to host **Gratitude** on a **Red Hat Enterprise Linux (RHEL)** server (RHEL 8 or 9 and compatible clones such as Rocky/Alma unless noted). It assumes SSH access (IP address, OS, username and password or key).

Contents

1	What you are hosting	2
2	Access the server	2
3	Install base software (RHEL)	2
3.1	Node.js 20+	2
3.2	PostgreSQL	3
3.3	Firewall (firewalld)	3
4	Deploy the application	3
4.1	Clone and install	3
4.2	API environment	3
4.3	Database migrate and seed	4
4.4	Build the web UI	4
4.5	Keep uploads on disk	4
5	Run the API as a service	4
6	nginx reverse proxy (RHEL paths)	5
6.1	SELinux notes	6
6.2	TLS (HTTPS)	6
7	Recommended env values	6
8	CI and CD (GitHub Actions)	6
8.1	Create an SSH deploy key	6
8.2	GitHub secrets	7
8.3	Server file for web build URL	7
9	Updates after go-live	7
10	Smoke checks	8
11	Common host failures (RHEL)	8

1 What you are hosting

Gratitude needs two parts on the server:

1. **API** - Node.js app in `api/` (port 3001 by default). Serves JSON, auth and uploaded card images under `/uploads`.
2. **Web** - Built React app from `web/`. Served as static files with nginx.

Also required:

- PostgreSQL database
- Durable disk for `api/uploads/`

One VM can also host other projects. Keep separate folders, databases and upload directories per app. Use nginx to route by hostname.

2 Access the server

1. Confirm you have: public IP, SSH username, password or SSH key.
2. On RHEL, try usernames in this order if IT did not specify one: `root`, `cloud-user`, `admin`.
3. From Windows PowerShell or Terminal:

```
ssh USERNAME@SERVER_IP
```

4. Confirm the OS:

```
cat /etc/os-release
```

5. Change the shared password on first login if it was emailed to you.
6. Prefer SSH keys for ongoing access. WinSCP is optional; SSH is enough.

3 Install base software (RHEL)

Use `dnf` (not `apt`). Enable needed repositories with your org's subscription or free mirrors as required.

```
sudo dnf -y update
sudo dnf -y install git curl tar gcc-c++ make nginx firewalld
sudo systemctl enable --now firewalld
```

3.1 Node.js 20+

NodeSource (example for RHEL 9; use the matching NodeSource script for your major version):

```
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf -y install nodejs
node -v
npm -v
```

If corporate policy blocks NodeSource, install Node from your approved internal RPM channel instead. Target Node 20 or newer.

3.2 PostgreSQL

Enable the PostgreSQL module (RHEL 8/9 style) or use your approved Postgres packages:

```
sudo dnf -y module reset postgresql
sudo dnf -y module enable postgresql:16
sudo dnf -y install postgresql-server postgresql
sudo postgresql-setup --initdb
sudo systemctl enable --now postgresql
```

If `postgresql:16` is unavailable, enable the highest version your RHEL release supports (for example `postgresql:15` or `postgresql:13`).

Create app role and database:

```
sudo -u postgres createuser grat WITH PASSWORD 'CHOOSE_A_STRONG_PASSWORD'
sudo -u postgres createdb project_grat -O grat
```

Allow local password auth for the app user. Edit `pg_hba.conf` (path is often under `/var/lib/pgsql/data/`) so local TCP uses `scram-sha-256` or `md5`, then:

```
sudo systemctl restart postgresql
```

Or point `DATABASE_URL` at a managed Postgres instance instead of installing on the VM.

3.3 Firewall (firewalld)

```
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

Do not expose port 3001 publicly if nginx proxies it on 80/443.

4 Deploy the application

4.1 Clone and install

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/SeniruR/project_grat.git
sudo chown -R $USER:$USER /var/www/project_grat
cd /var/www/project_grat

npm install
cd api && npm install
cd ../web && npm install
```

4.2 API environment

Create `/var/www/project_grat/api/.env`:

```
DATABASE_URL="postgresql://grat:PASSWORD@127.0.0.1:5432/project_grat?schema=public"
PORT=3001
HOST=0.0.0.0
JWT_SECRET="REPLACE_WITH_LONG_RANDOM_SECRET"
```

```
CORS_ORIGIN="https://YOUR_WEB_HOSTNAME"
PUBLIC_API_URL="https://YOUR_API_HOSTNAME"
AUTH_MODE=dev
DIRECTORY_MODE=mock
MAIL_MODE=mock
```

Notes:

- PUBLIC_API_URL must be the **API** public URL. Never point it only at a static frontend that cannot serve /uploads.
- CORS_ORIGIN must include the browser origin of the web UI.
- For real email later set MAIL_MODE=smtp or graph and the related secrets.

4.3 Database migrate and seed

```
cd /var/www/project_grat/api
npx prisma generate
npx prisma migrate deploy
npm run db:seed
```

4.4 Build the web UI

```
cd /var/www/project_grat/web
export VITE_API_URL="https://YOUR_API_HOSTNAME"
npm run build
```

Output is in web/dist/.

4.5 Keep uploads on disk

```
mkdir -p /var/www/project_grat/api/uploads
```

Back up this folder. If it is lost, card images break while database rows remain.

5 Run the API as a service

On RHEL, prefer a dedicated Linux user (example **grat**) rather than **www-data** (Debian-only name).

```
sudo useradd -r -s /sbin/nologin grat || true
sudo chown -R grat:grat /var/www/project_grat
```

Create /etc/systemd/system/gratitude-api.service:

```
[Unit]
Description=Gratitude API
After=network.target postgresql.service

[Service]
Type=simple
User=grat
Group=grat
WorkingDirectory=/var/www/project_grat/api
```

```
EnvironmentFile=/var/www/project_grat/api/.env
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now gratitude-api
sudo systemctl status gratitude-api
curl http://127.0.0.1:3001/health
```

6 nginx reverse proxy (RHEL paths)

On RHEL, site files usually go in `/etc/nginx/conf.d/` (there is typically no `sites-available` layout).

Example: web on `grat.example.com`, API on `api-grat.example.com`.

Create `/etc/nginx/conf.d/gratitude-web.conf`:

```
server {
    listen 80;
    server_name grat.example.com;
    root /var/www/project_grat/web/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Create `/etc/nginx/conf.d/gratitude-api.conf`:

```
server {
    listen 80;
    server_name api-grat.example.com;

    client_max_body_size 20M;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Test and reload:

```
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

6.1 SELinux notes

If nginx returns 502/403 with SELinux enforcing:

```
getenforce
sudo setsebool -P httpd_can_network_connect 1
sudo dnf -y install policycoreutils-python-utils
sudo semanage fcontext -a -t httpd_sys_content_t \
  '/var/www/project_grat/web/dist(/.*)?'
sudo restorecon -Rv /var/www/project_grat/web/dist
```

Only change SELinux settings your security team allows.

6.2 TLS (HTTPS)

When DNS points at the server:

```
sudo dnf -y install certbot python3-certbot-nginx
sudo certbot --nginx -d grat.example.com -d api-grat.example.com
```

Then set `CORS_ORIGIN`, `PUBLIC_API_URL` and `VITE_API_URL` to the `https://` URLs and rebuild the web app if needed.

7 Recommended env values

```
PUBLIC_API_URL=https://api-grat.example.com
CORS_ORIGIN=https://grat.example.com
VITE_API_URL=https://api-grat.example.com
```

8 CI and CD (GitHub Actions)

Continuous integration runs on every pull request and push (`.github/workflows/ci.yml`): API migrate and typecheck, web lint, typecheck and production build.

Continuous deployment runs after CI succeeds on `main` (`.github/workflows/cd-production.yml`). It SSHs to this RHEL host and runs `scripts/deploy-rhel.sh` (git sync, Prisma migrate, web build, restart API, reload nginx, local health check).

8.1 Create an SSH deploy key

GitHub Actions needs a key pair: the **private** key goes in a GitHub secret; the **public** key goes on the RHEL server. Generate the pair on your Windows PC (PowerShell). Leave the passphrase empty so Actions can use the key non-interactively.

```
ssh-keygen -t ed25519 -C "gratitude-github-deploy" -f $env:USERPROFILE\.ssh\
  gratitude_deploy
```

That creates `gratitude_deploy` (private) and `gratitude_deploy.pub` (public).

Install the public key on the server (same user as `DEPLOY_USER`, for example `cloud-user` or `root`). SSH in with a password once, then:

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
# Paste the one line from gratitude_deploy.pub into authorized_keys:
echo "PASTE_PUBLIC_KEY_LINE_HERE" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

From Windows you can also append the public key in one step if password SSH already works:

```
type $env:USERPROFILE\.ssh\gratitude_deploy.pub |  
ssh YOUR_USER@YOUR_SERVER_IP "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/  
authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Test passwordless login from your PC:

```
ssh -i $env:USERPROFILE\.ssh\gratitude_deploy YOUR_USER@YOUR_SERVER_IP
```

Never commit the private key. Never paste it into chat or tickets.

8.2 GitHub secrets

In the repository Settings under Secrets and variables for Actions, set:

- DEPLOY_HOST - server IP or hostname
- DEPLOY_USER - SSH user (root or cloud-user)
- DEPLOY_SSH_KEY - full contents of the private file `gratitude_deploy` (including the BEGIN/END lines)
- Optional: DEPLOY_PORT (default 22), DEPLOY_APP_ROOT (default `/var/www/project_grat`)

Create a GitHub Environment named `production` if you want a manual approval gate before each deploy.

8.3 Server file for web build URL

```
sudo mkdir -p /etc/gratitude  
sudo tee /etc/gratitude/deploy.env <<'EOF'  
VITE_API_URL=https://YOUR_API_HOSTNAME  
EOF  
sudo chmod 640 /etc/gratitude/deploy.env
```

The deploy user must be able to restart `gratitude-api` and reload `nginx` (passwordless `sudo` for those commands is typical).

9 Updates after go-live

Prefer the GitHub CD workflow above. For a manual update on the host:

```
cd /var/www/project_grat  
bash scripts/deploy-rhel.sh
```

Or step by step:

```
cd /var/www/project_grat  
git pull  
cd api && npm install && npx prisma migrate deploy  
cd ../web && npm install  
export VITE_API_URL="https://YOUR_API_HOSTNAME"  
npm run build  
sudo systemctl restart gratitude-api  
sudo systemctl reload nginx
```

Do not delete `api/uploads/` during updates.

10 Smoke checks

1. `curl https://YOUR_API_HOSTNAME/health` returns OK.
2. Open the web URL and sign in.
3. Upload or import a card and confirm images appear after refresh.
4. Restart the API (`sudo systemctl restart gratitude-api`) and confirm images still load.
5. Send a test card (mock or SMTP as configured).

11 Common host failures (RHEL)

Symptom	Fix
Cannot SSH	Check IP, username (<code>root/cloud-user</code>), port 22 and firewall.
<code>dnf</code> package missing	Enable correct RHEL repos / subscription; ask IT for approved Node/Postgres RPMs.
API migrate fails	Fix <code>DATABASE_URL</code> ; check Postgres is running and <code>pg_hba.conf</code> allows the app user.
nginx 502 Bad Gateway	API down, or SELinux blocking proxy - enable <code>httpd_can_network_connect</code> .
Broken card images	Uploads folder missing; restore backup or re-upload. Check <code>PUBLIC_API_URL</code> .
Browser CORS errors	Add web origin to <code>CORS_ORIGIN</code> ; restart API.
Web loads but API fails	Wrong <code>VITE_API_URL</code> at build time; rebuild web.